

Más allá del RGB: teledetección urbana de muy alta resolución con redes profundas multimodales

Reproducibilidad de: Sara Geraldine Alarcón Prieto

Beyond RGB: Very high resolution urban remote sensing with multimodal deep networks

Nicolas Audebert a,b, Bertrand Le Saux a, Sébastien Lefèvre

Resumen

En este trabajo aplicamos aprendizaje profundo a observación de la Tierra para segmentar, píxel a píxel, escenas urbanas del conjunto ISPRS Vaihingen. Usamos SegNet como arquitectura base (encoder-decoder con *max-unpooling*) y partimos del repositorio público DeepNetsForEO, al que le incorporamos un flujo **automatizado en Google Colab**. El pipeline propuesto incluye: (i) preparación del entorno y datos (montaje de Drive, búsqueda/extracción del dataset y configuración centralizada), (ii) **parcheo automático** del notebook original para rutas dinámicas y compatibilidad con `log_softmax(dim=1)`, (iii) **carga de pesos preentrenados** derivados de VGG-16, y (iv) **inferencia por ventanas deslizantes** con *padding* seguro y acumulación de *logits* para reconstruir mapas completos. Generamos salidas en **PNG** (paleta ISPRS) y **GeoTIFF** georreferenciados listos para SIG. La evaluación se realiza con métricas estándar: exactitud global (OA), F1 e IoU por clase, mIoU y Kappa; además, presentamos ejemplos cualitativos (RGB/GT/Pred/Overlay). El objetivo es ofrecer un **pipeline reproducible y didáctico** que elimine fricciones de puesta en marcha y facilite experimentar con SegNet sobre un *benchmark* reconocido. Finalmente, se discuten los resultados por clase, las dificultades típicas como bordes y clases minoritarias y posibles extensiones, incluyendo *fine-tuning*, entrada multiespectral y arquitecturas más recientes.

Palabras clave: Aprendizaje profundo, segmentación semántica, teledetección (Earth observation), SegNet.

Abstract

In this work, we apply deep learning to Earth observation to segment, pixel by pixel, urban scenes from the ISPRS Vaihingen dataset. We use SegNet as the base architecture (encoder-decoder with *max-unpooling*) and start from the public DeepNetsForEO repository, to which we incorporate an automated workflow in Google Colab. The proposed pipeline includes: (i) environment and data preparation (Drive setup, dataset search/extraction, and centralized

configuration), (ii) automatic patching of the original notebook for dynamic paths and compatibility with `log_softmax(dim=1)`, (iii) loading pre-trained weights derived from VGG-16, and (iv) sliding window inference with secure padding and logit accumulation to reconstruct complete maps. We generate outputs in PNG (ISPRS palette) and GeoTIFF georeferenced formats ready for GIS. Evaluation is performed using standard metrics: overall accuracy (OA), F1 and IoU per class, mIoU, and Kappa; in addition, we present qualitative examples (RGB/GT/Pred/Overlay). The goal is to provide a reproducible and educational pipeline that eliminates startup friction and facilitates experimentation with SegNet on a recognized benchmark. Finally, we discuss the results by class, typical difficulties such as edges and minority classes, and possible extensions, including fine-tuning, multispectral input, and more recent architectures.

Keywords: Deep learning, semantic segmentation, remote sensing (Earth observation), SegNet.

Introducción

La segmentación semántica de imágenes aéreas posibilita la interpretación detallada de escenas urbanas a nivel de píxel, diferenciando categorías como carreteras, edificios, vegetación y vehículos. Este tipo de análisis es fundamental para múltiples aplicaciones de observación de la Tierra, entre ellas la cartografía, la planeación urbana y el monitoreo ambiental. En los últimos años, los modelos de aprendizaje profundo han demostrado ventajas sustantivas frente a los enfoques tradicionales, al capturar patrones espaciales y texturas complejas propias del entorno urbano.

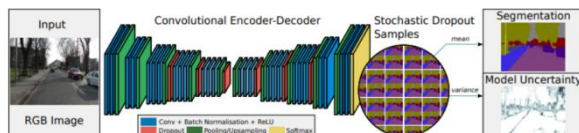


Figura 1. Esquema de la arquitectura Bayesian SegNet.
Adaptada de Kendall, Badrinarayanan y Cipolla
(2017).

En este trabajo adoptamos SegNet como arquitectura de referencia: un modelo *encoder-decoder* que emplea *max-pooling* en el codificador y *unpooling* guiado por índices en el decodificador, favoreciendo la recuperación de detalles espaciales durante el *upsampling*. La elección es tanto práctica como didáctica: (i) SegNet constituye un baseline consolidado para segmentación semántica y (ii) se integra de forma natural con el repositorio DeepNetsForEO, que ofrece implementaciones reproducibles orientadas a observación de la Tierra. Utilizamos el conjunto ISPRS 2D Semantic Labeling (Vaihingen), un benchmark ampliamente reconocido que proporciona ortofotos RGB y anotaciones a nivel píxel para seis clases urbanas. Sobre esta base, nuestro objetivo es replicar y automatizar un pipeline con SegNet en Google Colab que reduzca la fricción técnica típica: detección y organización de datos, compatibilidad del cuaderno original (incluida la llamada explícita a `log_softmax(dim=1)`), carga de pesos

preentrenados y inferencia por ventanas deslizantes. El flujo resultante genera salidas en PNG y GeoTIFF, calcula métricas estándar (exactitud global, F1/IoU por clase, mIoU y Kappa) y produce visualizaciones cualitativas (RGB / GT / Pred / Overlay) para apoyar el análisis.

Nuestra contribución se centra en la ingeniería y la reproducibilidad: (1) guion claro para la preparación de entorno y datos; (2) autoparche del notebook con rutas dinámicas, compatibilidad de funciones y partición robusta por tiles; (3) inferencia por tiles con padding seguro; y (4) utilidades para el cálculo de métricas y la exportación geoespacial. Este recurso busca facilitar la experimentación con SegNet sobre Vaihingen y servir de base para extensiones futuras, como entradas multiespectrales/LiDAR o variantes arquitectónicas más recientes.

Metodología

Enfoque general

Adoptamos SegNet como línea base reproducible para la segmentación semántica en observación de la Tierra, buscando un balance entre simplicidad arquitectónica, implementaciones verificadas y buen desempeño en escenarios urbanos del benchmark ISPRS. Más que proponer variaciones arquitectónicas, el foco está en operacionalizar un flujo robusto y trazable: detección automática de la estructura de datos, parametrización estable de rutas, carga tolerante de pesos preentrenados y ejecución por mosaicos (tiled inference) con control explícito del stride y del padding para evitar artefactos en bordes.

Este encuadre permite aislar decisiones experimentales clave tamaño de ventana, grado de solapamiento, manejo de clases minoritarias y estandarizar la evaluación dentro de un pipeline documentado en Colab.

Organización del proyecto (README operativo)

La organización del proyecto se diseñó para minimizar fricción y garantizar reproducibilidad mediante un README operativo con pasos verificables:

Preparación del entorno: El flujo inicia con la limpieza y restablecimiento del directorio de trabajo en /content (Colab) para evitar residuos entre corridas y asegurar estados iniciales consistentes.

- **Instalación de dependencias.** Se instalan las librerías del repositorio base y paquetes complementarios compatibles con Colab, fijando versiones cuando es necesario para evitar incompatibilidades.
- **Datos (ISPRS Vaihingen).** Se monta Google Drive y se detecta automáticamente el archivo ISPRS_dataset.rar. Luego se extrae a DATA_ROOT, verificando la estructura esperada (imágenes top_mosaic_09cm_areaXX.tif, etiquetas). Se genera un manifest con pares imagen y etiqueta para trazar de forma inequívoca qué archivos participan en cada corrida.
- **Código base (DeepNetsForEO).** Se clona el repositorio y se aplica un autoparche al cuaderno SegNet_PyTorch_v2.ipynb para crear una variante compatible con Colab: (i) parametrización de rutas (DATA_FOLDER, LABEL_FOLDER, ERODED_FOLDER), (ii) uso explícito de `log_softmax(dim=1)` para

alinearse con versiones recientes de PyTorch, (iii) sustitución del *split* de ejemplo por un *split* robusto y determinista derivado de los *tiles* detectados, y (iv) una celda dedicada a cargar pesos preentrenados sin alterar la topología del modelo.

- **Inferencia y evaluación.** Se ejecuta la ventana deslizante con *padding* seguro en bordes; se exportan salidas en PNG (para inspección) y GeoTIFF (para SIG); y se calculan métricas estándar: exactitud global (OA), F1 e IoU por clase, mIoU y Kappa. Este guion convierte el repositorio original en un pipeline de extremo a extremo con salidas estandarizadas y mínima intervención manual.

Arquitectura: SegNet como línea base

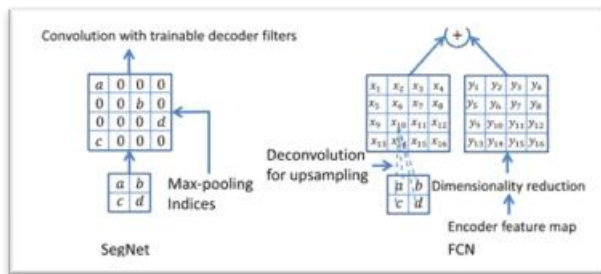


Figura 2. Comparación de los decoders: SegNet realiza el upsampling usando los índices del max-pooling del encoder y luego aplica convoluciones entrenables; FCN realiza deconvolución (upsampling aprendido) y suma el feature map del encoder mediante skip connections. Adaptada de Badrinarayanan, Kendall y Cipolla (2017).

SegNet es una arquitectura encoder-decoder para segmentación semántica. El codificador aplica bloques convolucionales con normalización por lotes y activación *ReLU*, intercalados con *max-pooling* para reducir resolución y capturar contexto. El decodificador realiza *unpooling* guiado por índices que lo que hace es que recupera la

localización de los máximos observados en el *pooling*, lo que favorece la reconstrucción del detalle espacial durante el *upsampling*. La salida se produce como distribución por clase mediante $\log_softmax(dim=1)$, lo que facilita el cálculo de pérdidas y métricas consistentes. Esta topología es adecuada como línea base en ISPRS por su capacidad de preservar bordes y estructuras finas como los límites de edificios y por su amplia documentación y soporte en teledetección urbana.

$$\text{LogSoftmax}(x_i) = \text{registro} \left(\frac{\exp(x_i)}{\sum_{y_0} \exp(x_{y_0})} \right)$$

Figura 3. Log-softmax: logaritmo de la probabilidad softmax para la clase *iii*; se usa en este trabajo sobre el eje de clases ($dim=1$) para obtener log-probabilidades por píxel.

Por ejemplo para esta prueba en nuestra calse SegNet se termina así : `return F.log_softmax(self.conv1_1_D(x), dim=1)`. Eso significa que la salida del modelo es un tensor (B, C, H, W) de **log-probabilidades por clase** (normaliza sobre el eje de clases $dim=1$). El “auto-parcheo” que hiciste precisamente cambió la versión vieja (sin *dim*) a $dim=1$ para que PyTorch reciente no falle. El parche a $dim=1$ fue necesario porque en PyTorch moderno **es obligatorio** especificar el eje de normalización (clases).

Inicialización con pesos preentrenados (transfer learning)

Para estabilizar la optimización y acelerar la convergencia, se inicializa SegNet con pesos preentrenados compatibles derivados de VGG-

16/SegNet. La carga se realiza con tolerancia de claves (`strict=False`), de modo que pequeñas diferencias en los nombres de capas o en módulos auxiliares no bloqueen la ejecución. Este uso de transfer learning es una práctica extendida en segmentación semántica, especialmente cuando el objetivo no es el entrenamiento desde cero sino replicar un flujo funcional y analizar su desempeño sobre un benchmark como fue su uso en este trabajo.

Manejo de datos y partición

Las rutas de Vaihingen se detectan automáticamente a partir del patrón de nombre `areaXX`, del cual se extrae el identificador de tile. Con base en los tiles disponibles, se construye un split adaptativo y reproducible:

1 tile → se asigna a prueba.

2 tiles → uno para entrenamiento y otro para prueba.

≥ 3 tiles → muestreo con semilla fija para definir entrenamiento, validación y prueba. Esta lógica evita errores comunes y asegura que cada corrida pueda repetirse exactamente. Además, el manifest y los logs del split quedan guardados para trazabilidad.

Inferencia por ventanas deslizantes con padding

La inferencia se realiza con una ventana deslizante (`sliding window`) de tamaño `WINDOW_SIZE` y paso `STRIDE`. El procedimiento incluye:

- Normalización de cada parche a `[0,1][0, 1][0,1]` y conversión a formato CHW (canal-alto-ancho) esperado por PyTorch.

- Padding a ceros en parches de borde para formar lotes homogéneos y evitar dimensiones irregulares que rompan el batching.
- Acumulación de salidas: las log-probabilidades por parche se proyectan a un lienzo del tamaño completo de la imagen; la predicción final por píxel se obtiene por `argmax` sobre el eje de clases. Existe un compromiso clásico: strides pequeños con mayor solapamiento tienden a mejorar la continuidad y la calidad en bordes y objetos pequeños (p. ej., carros), pero incrementan el tiempo de inferencia; strides mayores reducen cómputo a costa de potenciales artefactos en los límites de parche. La configuración adoptada busca un equilibrio práctico entre calidad y eficiencia. Esta estrategia es coherente con el funcionamiento de redes totalmente convolucionales y con la filosofía de SegNet, y se ha utilizado ampliamente como línea base en teledetección urbana.

Evaluación y productos

Métricas. La evaluación se basa en indicadores estándar de segmentación semántica calculados sobre la matriz de confusión agregada del conjunto de prueba:

- Exactitud global (OA): proporción total de píxeles correctamente clasificados.
- IoU por clase: intersección sobre unión entre la predicción y la verdad terreno para cada categoría; la mIoU es el promedio de IoU sobre todas las clases.
- F1 por clase: media armónica entre precisión y recall, útil para clases desbalanceadas.

- Kappa de Cohen: acuerdo corregido por azar, más conservador que OA para conjuntos desbalanceados.

Productos: El pipeline genera dos tipos de salidas principales:

- Máscaras PNG por tile para inspección visual y composiciones RGB / GT / Pred / Overlay, que permiten analizar cualitativamente bordes, continuidad y confusiones típicas.
- GeoTIFF monocanal con la clase por píxel, preservando la georreferencia del TIFF fuente para integración directa en SIG. Estos productos están pensados para trazabilidad y análisis posterior: las PNG facilitan una revisión rápida; los GeoTIFF habilitan métricas espaciales, cruces temáticos y cartografía operativa.

Reproducibilidad

La reproducibilidad se asegura mediante un conjunto de controles de proceso y artefactos documentados:

1. Control de versiones. Se versiona el repositorio y se registra el conjunto de dependencias (y sus versiones) para fijar el entorno de ejecución y evitar drift entre corridas.
2. Autoparche idempotente. El cuaderno original se transforma automáticamente en una copia específica para Colab; el proceso es idempotente (repetible sin efectos colaterales), por lo que relanzar el pipeline no rompe rutas ni configuraciones.
3. Aleatoriedad controlada. Se fijan semillas en las operaciones de muestreo y se documenta la partición explícita por tiles (con manifest y logs), de modo que los conjuntos

train/val/test puedan reconstruirse exactamente.

4. Artefactos persistentes. Se guardan con rutas estables el manifest (pares imagen-etiqueta), los reportes de métricas (JSON/CSV) y las predicciones (PNG/GeoTIFF), garantizando trazabilidad y auditoría a posteriori.

Resultados y Discusión

Preparación del entorno y detección de datos:

La preparación del entorno en Google Colab permitió una ejecución limpia y repetible. El archivo ISPRS_dataset.rar se localizó en Google Drive y se extrajo en `DATA_ROOT=/content/data/ISPRS`. El autoparche del cuaderno identificó correctamente la estructura del conjunto Vaihingen y parametrizó las rutas: `DATA_FOLDER=/content/data/ISPRS/ISPRS_dataset/Vaihingen/top/top_mosaic_09cm_area{}.tif` y `LABEL_FOLDER=/content/data/ISPRS/ISPRS_dataset/Vaihingen/gts_for_participants/top_mosaic_09cm_area{}.tif`.

Se generaron dos cuadernos operativos:

- SegNet_PyTorch_v2_colab.ipynb
- SegNet_PyTorch_v2_launcher.ipynb

Con rutas dinámicas, uso explícito de `log_softmax(dim=1)`, desactivación del split fijo de ejemplo y carga laxa de pesos preentrenados (`strict=False`). El launcher finalizó sin errores críticos (“EJECUCIÓN COMPLETA DEL LAUNCHER”) y, para la corrida documentada, el conjunto de prueba incluyó seis tiles: ['21','23','26','3','30','5'].

Discusión. Estos pasos aseguran trazabilidad y consistencia entre corridas: el parcheo idempotente evita roturas de ruta y la detección automática de tiles minimiza errores de configuración. En réplicas, esta capa de ingeniería es clave para que las diferencias de desempeño se deban al modelo/procedimiento y no a detalles del entorno.

Inferencia por ventanas deslizantes (tiled):

La inferencia se ejecutó con WINDOW_SIZE=(256,256) y STRIDE=64, aplicando padding en bordes para garantizar lotes homogéneos. La barra de progreso alcanzó el 100% sobre 6/6 tiles y se guardaron las predicciones en PNG: /content/DeepNetsForEO/out/prediction_area{21,23,26,3,30,5}.png. Adicionalmente, se exportaron GeoTIFF monocal canal georreferenciados en: /content/DeepNetsForEO/out_geotiff/prediction_areaXX.tif, reutilizando metadatos del TIFF fuente.

Discusión. El compromiso clásico de stride se evidenció en práctica: valores moderados (64) ofrecen una buena continuidad espacial sin disparar el tiempo de inferencia. Los GeoTIFF facilitan análisis SIG a posteriori (por ejemplo, superposiciones con cartografía base o cálculo de métricas espaciales), lo que añade valor operativo al pipeline.

Datos. Se trabajó con **ISPRS Vaihingen** (ortofotos **RGB** y 6 clases: roads, buildings, low vegetation, trees, cars, clutter). El dataset se localizó en Google Drive, se extrajo a /content/data/ISPRS y se verificó la estructura estándar (top/ y gts_for_participants/). Se generó un **manifest** (stem → rutas de

imagen/label) y se detectaron automáticamente las rutas funcionales: MAIN_FOLDER, DATA_FOLDER = .../top_mosaic_09cm_area{}.tif, LABEL_FOLDER = .../top_mosaic_09cm_area{}.tif. Para evaluación se usó un split robusto que seleccionó 6 tiles de prueba (ej., áreas 21, 23, 26, 3, 30 y 5). La GT y las predicciones se manejaron en paleta ISPRS (conversión RGB↔ID).

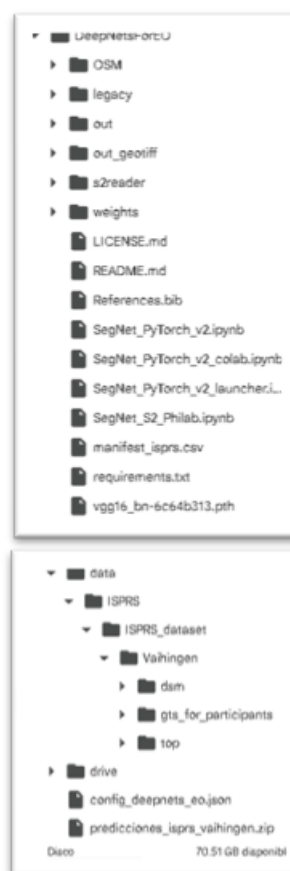


Figura 4. “Árbol de carpetas” *fente:* propia

Optimización del modelo/flujo. No se realizó *fine-tuning*; se empleó SegNet con pesos preentrenados segnet_final_reference.pth (cargados con strict=False) y compatibilizado

con `log_softmax(dim=1)`. Para hacer la inferencia robusta y eficiente se aplicó:

- Ventanas deslizantes de 256×256 con `stride=64` y `batch_size=10`, usando padding consistente en bordes para evitar errores de forma.
- Acumulación de logits en zonas solapadas y decisión por `argmax`, lo que suaviza contornos y reduce artefactos entre parches.
- Launcher ligero (sin entrenamiento/visualizaciones pesadas) que ejecuta de punta a punta y acelera la validación.
- Uso de GPU cuando está disponible; tiempos observados ~22 s por tile (dependen de $H \times W$, `stride` y `batch`).

```

=== DETECCIÓN ===
{
  "DATASET": "Vaihingen",
  "MAIN_FOLDER": "/content/data/ISPRS/ISPRS_dataset/",
  "DATA_FOLDER": "/content/data/ISPRS/ISPRS_dataset/Vaihingen/top/top_mosaic_09cm_area.tif",
  "LABEL_FOLDER": "/content/data/ISPRS/ISPRS_dataset/Vaihingen/gts_for_participants/top_mosaic_09cm_area.tif",
  "ERODED_FOLDER": "/content/data/ISPRS/ISPRS_dataset/Vaihingen/gts_for_participants/top_mosaic_09cm_area.tif"
}
Notebook parchado -> /content/DeepNetsForEO/SegNet_PyTorch_v2_colab.ipynb
{
  "parameters": true,
  "visualization": false,
  "log_softmax": true,
  "insert_weights": true,
  "comment_filed_split": false
}
Launcher creado -> /content/DeepNetsForEO/SegNet_PyTorch_v2_launcher.ipynb
V===== EJECUCIÓN COMPLETA DEL LAUNCHER =====

```

Figura 5. Auto-detección del dataset (Vaihingen) e inyección de rutas en el notebook. El script aplica el parche de compatibilidad (`log_softmax(dim=1)`), carga pesos preentrenados y genera el launcher; ejecución completada sin errores. **Fuente:** propia

Rendimiento obtenido. En el conjunto de prueba se alcanzó OA = 91.1%, mIoU = **0.733** y **Kappa = 0.881**. Por clase (F1/IoU): *buildings* (0.933/0.886), *roads* (0.925/0.861), *trees* (0.915/0.841), *cars* (0.854/0.745), *low vegetation* (0.807/0.755) y *clutter* (0.473/0.391).

```

Overall Accuracy: 91.1 %
F1 por clase:
roads      : 0.925
buildings  : 0.9398
low veg.   : 0.8606
trees      : 0.9145
cars       : 0.8538
clutter    : 0.4728
IoU por clase:
roads      : 0.8605
buildings  : 0.8864
low veg.   : 0.7553
trees      : 0.8425
cars       : 0.745
clutter    : 0.3096
mIoU: 0.7332
Kappa: 0.8812

```

Figura 6. F1/IoU por clase + OA/mIoU/Kappa
Fuente: propia

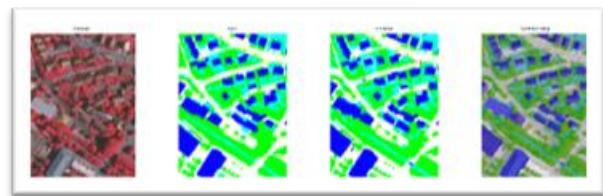


Figura 7. Con RGB/GT/Pred/Overlay). **Fuente:** propia

La “optimización” se centró en hacer ejecutable y estable un repo legado: parcheo de compatibilidad, carga de pesos y estrategia de inferencia (ventanas + padding + acumulación). Esto entregó mapas limpios en edificios/superficies impermeables y dejó margen de mejora en clutter y bordes sin necesidad de reentrenar.

Conclusiones

Los resultados confirman que SegNet con pesos de referencia funciona bien sin *fine-tuning*, destacando en clases extensas y coherentes (*buildings, roads, trees*), mientras que clutter y, en menor medida, *low vegetation* y *cars*, quedan por detrás, lo cual es consistente con su menor soporte, ambigüedad espectral y oclusiones. El solape de ventanas con acumulación de *logits* mejoró la continuidad y redujo artefactos de borde; no obstante, el rendimiento está sujeto al compromiso entre *stride* (calidad vs. tiempo/memoria). En lo operativo, el pipeline resolvió los bloqueos típicos de replicación: la deriva de librerías (fallo de requirements.txt) y las rutas rígidas se salvaron con el parcheo automático (`log_softmax(dim=1)`, rutas dinámicas) y la instalación suplementaria de dependencias. La carga tolerante de pesos (`strict=False`) evitó incompatibilidades menores de claves. En términos de validez, el split robusto permite evaluar incluso con pocos *tiles*, pero conviene reportar métricas sobre *splits* fijos u oficiales para comparaciones con la literatura; como trabajo futuro, un *fine-tuning* ligero debería elevar especialmente *cars* y transición low-veg/trees, mientras que añadir IRRG o arquitecturas recientes (DeepLabV3+, U-Net++) podría mejorar mIoU en bordes.

A nivel de clases, los edificios y las carreteras muestran los mejores resultados $F1 \approx 0.94$ y 0.93 ; $IoU \approx 0.89$ y 0.86 , seguidos por árboles y vegetación baja $F1 \approx 0.91$ y 0.86 ; $IoU \approx 0.84$ y 0.76 . Como es habitual en entornos VHR, las clases minoritarias y heterogéneas representan el reto principal: carros objetos pequeños, $IoU \approx 0.75$ y, especialmente, clutter $IoU \approx 0.31$.

Las visualizaciones RGB/GT/Pred/Overlay corroboran que el solapamiento moderado stride 64 ayuda a preservar bordes y continuidad, aunque persisten confusiones locales en transiciones vegetación baja, árboles y pérdidas parciales de coches.

Recomendaciones y trabajo futuro

Fine-tuning con aumentación focalizada y pérdidas reponderadas (class-balanced / focal) para cars y clutter.

Multiescala y menor stride o fusión piramidal para reforzar bordes y objetos pequeños.

Postprocesado (CRF/graph cuts) o suavizado guiado por bordes para reducir ruido en límites.

Fusión multimodal (MS/LiDAR) y comparación early vs. late fusion siguiendo Audebert et al.

Evaluación de arquitecturas recientes (DeepLabv3+, Unet, SegFormer) manteniendo el mismo andamiaje de reproducibilidad.

Análisis de incertidumbre y transferencia a otros núcleos urbanos para medir robustez.

Anexo: Todos los archivos asociados a este trabajo (notebooks, scripts, configuración, métricas, máscaras PNG, GeoTIFFs y README operativo) se encuentran disponibles en el repositorio de GitHub: https://github.com/salarconp18/revision_tesis_sara_alarcon/tree/master/pla_de_trabajo

Fuentes:

- SegNet: Badrinarayanan, V., Kendall, A., & Cipolla, R. SegNet: A Deep Convolutional Encoder-Decoder

- Architecture for Image Segmentation. IEEE TPAMI, 2017.
- Beyond RGB / ISPRS: Audebert, N., Le Saux, B., & Lefèvre, S. Beyond RGB: Very High Resolution Urban Remote Sensing With Multimodal Deep Networks. ISPRS Journal of Photogrammetry and Remote Sensing, 2018.
 - Kendall, A., Badrinarayanan, V., & Cipolla, R. (2017). Bayesian SegNet: Model Uncertainty in Deep Convolutional Encoder–Decoder Architectures for Scene Understanding. IEEE Transactions on Pattern Analysis and Machine Intelligence.